

Porting the game interpreter and compiler for the 6800 to the 6809

 flexonsbd.blogspot.com/2021/05/6800game6809.html

Wednesday, May 19, 2021

Porting the 6800 game interpreter and compiler to the 6809.

I ported not only the game interpreter for the 6800, but also the compiler.

motive

Since the printed circuit boards I ordered still haven't arrived, I've been spending most of my time tinkering with software lately. Last time, I managed to port the TL1 compiler for the 6800 to the 6809, so I decided to ride that momentum and try porting the game interpreter and compiler this time.

Ports of the GAME interpreter to the 6809 already existed, such as Makoto Fujiwara's "GAME09 Interpreter" in 1981 and Tetsuya Hagihira's "GAME-FM Interpreter Compiler" in 1984. Both were excellent in terms of execution speed and functionality, and I myself installed and used them. However, the source code for both was not publicly available; only binary code was available.

The compiler to be ported is

The compiler I'm planning to port this time is the "GAME68 compiler" that I introduced in my blog post "Running the GAME68 compiler on a 6802 board" on May 8, 2019, and it was written by Yoshiaki Matsushima himself at GAME.

The fact that it's written within the game itself means that the compiler's logical structure is the same for both the 6800 and 6809, and only the code generation part differs.

The generated code is a 6800 binary, but as I mentioned in the previous TL1, there aren't that many significant differences between the 6809 and 6800 binaries, so I thought I could just rewrite the parts that were different. (A bit simplistic, I know.)

Looking at the compiler's runtime routines

First, I looked into the runtime routines.

Because it utilizes routines within the GAME interpreter, it's a small file of less than 300 bytes, but when I disassembled it, I found that the only instructions that differed from the 6809 were NOP and LDX, so I was able to rewrite it easily.

However, since it uses routines within the interpreter, it means that the 6809 version of the interpreter is required at runtime, which means that the interpreter needs to be ported before the compiler...

The interpreter needed to be ported.

So, we decided to start by porting the interpreter.

The assembler source code for the 6800 interpreter is also publicly available, just like with the TL1. Therefore, by rewriting it while paying attention to stack instructions, just as I did with the TL1, I was able to get it working without too much difficulty. (In my case, I don't particularly care about speed in either hardware or software; as long as it works correctly, that's good enough, so I haven't done much optimization.)

The interpreter porting is complete.

Now that the interpreter porting is complete and the addresses of the routines called by the compiler's runtime routines have been determined, I created a version of GAME68 where the compiler-related addresses written within the GAME68 itself have been rewritten, and examined the code generated by a simple program.

Then, I discovered that there are programs that work simply by rewriting the X register-related instructions and the PSH and PUL instructions.

Encouraged by these results, I took a look at the compiler source code and found that among the instructions generated by the compiler, there were nine types whose codes differed between the 6800 and 6809: LDX, STX, INX, TAB, TBA, PSHA, PSHB, PULA, and PULB. Although these differ in the number of bytes, replacing them does not seem to be too difficult. ~~However, it appears that some stack-related instructions, LDS and STS, are also used, so caution is needed with these.~~

[Added May 23] Correction: The LDS and STS instructions were not used.

I tried making a GAME09 compiler (or something similar).

For now, I've left the stack-related instructions as they are (since I don't yet know which GAME instructions use them...) and created a version where only the other instructions have

been replaced.

Detailed operating environment

The 6809 board used is the same 6809/6802 DualCPU board as last time.



6809/6802 Dual CPU board

This is a memory map.

0000	インタプリタ ワークエリア
0100	GAME9インタプリタ (拡張機能付き)
0A00	
2100	GAME9コンパイラ ランタイムルーチン
22BD 22BE	生成オブジェクト
3000	ソースプログラム
4000	GAME9コンパイラ (GAME自身で記述)
6200	

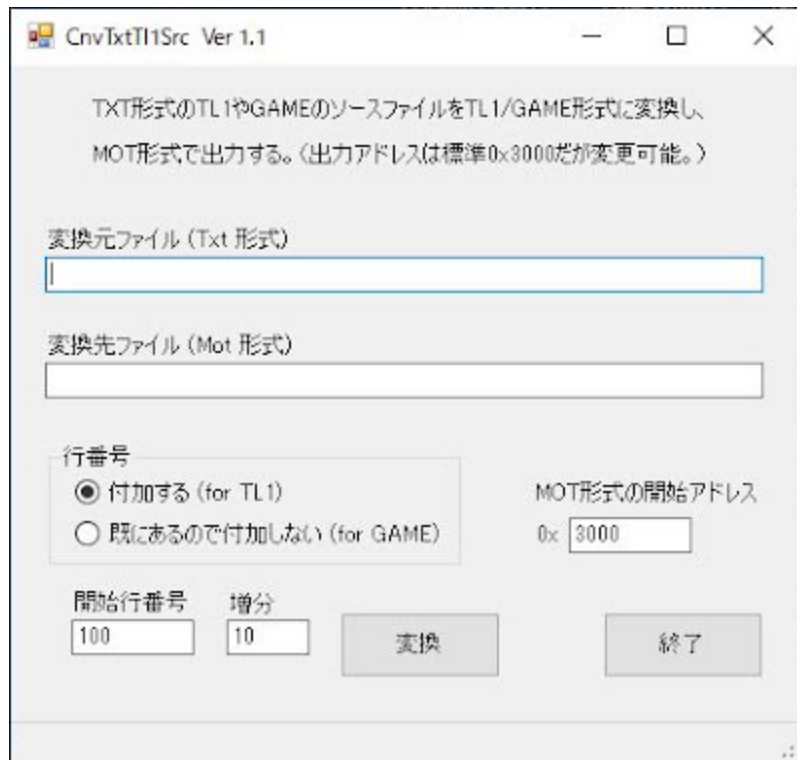
Memory map

The storage area for source programs starts at \$3000, the same as for the TL1 compiler.

The original runtime routine is \$2100~\$22BA, but a jump instruction (3 bytes) to the created object has been added to the beginning, allowing it to be executed with JMP \$2100.

To load each program, I first convert them to MOT format and then load them using the L command on my custom monitor or the LOAD command on ASSIST09.

Regarding the program for converting to MOT format, while the previous TL1 compiler source code was converted from a file created without line numbers in the editor, the GAME source code has line numbers, so the program has been enhanced to convert it to binary and add new storage addresses. It also now allows specifying the line numbers to add and the storage addresses.



CvtTxtTl1Src.exe

Compilation execution steps

Compilation is performed using the following steps:

1. Load the GAME9 interpreter on the monitor.

L GAME9EX

GAME9 with extensions is available from \$0.100.

2. Start the interpreter (cold start)

G 100

3. Move to the monitor.

>=\$F800

4. Load the source program.

L (source file)

The source program is loaded starting from \$3000.

5. Load the GAME9 compiler and runtime routines.

L GAME9CS

Runtime routines are loaded starting at \$2100, and the compiler at \$4000.

6. Return to the GAME9 interpreter (with hot start).

G 103

7. Activate the compiler.

=\$4000 (Set of starting addresses)

== (Searching for the last address)

8. Execute compilation.

#=1

Example

Here's an example of how it works. In the original, compilation starts after answering several queries, such as the work area, but since we've hardcoded the source program's storage area, compilation starts immediately with #=1.

```

1 9070 :=G:0)=0 #=700↓
2 9080 "Z"↓
3 9999 "ER" #=-1↓
4 ↓
5 *READY↓
6 :#=1↓
7 ↓
8 GAME SRC=3000↓
9 48 BYT (10LINE)↓
10 SUB PRG=2103 END=222D↓
11 OBJ START=222E↓
12 WORK=1204↓
13 STACK=1500↓
14 70 222E 3000↓
15 80 222E 3021↓
16 90 222E 302E↓
17 100 222E 304F↓
18 110 2242 305E↓
19 120 225A 306D↓
20 130 227A 307C↓
21 140 228A 3086↓
22 150 22BF 30A0↓
23 160 22DF 30B1↓
24 70 222E 3000↓
25 80 222E 3021↓
26 90 222E 302E↓
27 100 222E 304F↓
28 110 2242 305E↓
29 120 225A 306D↓
30 227A↓
31 130 227A 307C↓
32 140 228A 3086↓
33 150 22BF 30A0↓
34 2242↓
35 160 22DF 30B1↓
36 SIZE: 177(222E-22DE)↓
37 END↓
38 *READY↓
39 :

```

Example

After compilation is complete, running `>=\$2100` in the GAME will execute the generated object.

In conclusion

I haven't yet replaced all the 6800 instructions with 6809 instructions, but I've compiled several programs and confirmed that they work correctly.

When I previously wrote a blog post about the "GAME68 Compiler," I received permission from the compiler's creator, Yoshiaki Matsushima, to publish it. Therefore, I think it's okay to publish this version, which has been rewritten for the 6809.

I doubt there are any eccentric people who would want to use GAME or TL1 in this day and age, but if there is any interest, I plan to release them once I have finished rewriting the stack-related instructions that have remained unchanged. The same applies to the conversion program CvtTxtTI1Src.exe.

The above is just something I did on a whim, and it has almost no practical use, but there is a common perception that the 6800 and 6809 are not binary-level compatible, etc. However, they are not entirely different, so I hope this serves as an example that you can do things like this if you pay attention to the differences. (Although, these days, even 6809 programs, let alone 6800 programs, are like relics of the fossil age...)